

STUDENT MANAGEMENT SYSTEM

A console-based academic record management application built with Python, JSON persistence, validation and reporting.

Author	HARSH VARDHAN
Course	B.Tech CSE (AIML)
Application Type	Console-based Python Application
Core Technologies	Python 3 • JSON • Standard Library
Development Practices	Validation • Exception Handling • Git • GitHub

PROJECT REPORT

A practical implementation of CRUD operations, academic performance analysis, persistent data storage and defensive input handling.

Prepared for academic evaluation and repository documentation

Contents

No.	Section	Focus
01	Executive Summary	Purpose, scope and project snapshot
02	Problem & Objectives	Need, scope and intended outcomes
03	Requirements & Features	Functional and non-functional requirements
04	System Architecture	Application flow, layers and data movement
05	Implementation	Storage, validation, student and marks modules
06	Academic Logic	Total, percentage, grade, result and performance message
07	User Workflow	Menu, CRUD and reporting workflows
08	Reliability & Error Handling	JSON safety, validation and failure handling
09	Testing & Verification	Test strategy and evidence recommendations
10	Design Decisions & Learning	Engineering rationale and outcomes
11	Future Roadmap	GUI, database, authentication and analytics
12	Conclusion	Final assessment of the project

Source alignment: This report is based primarily on the uploaded README, project statement and source code. The source code is the authority for the implemented behavior; where the README describes a modular repository while the supplied final code is a single-file version, that distinction is stated explicitly rather than quietly pretending the two are identical.

01 • Executive Summary

The **Student Management System using Python** is a console application designed to manage student records and academic performance. Its core workflow covers student creation, viewing, searching, updating and deletion, followed by marks entry and performance reporting.

The supplied implementation is deliberately lightweight: it uses Python's standard library and stores persistent records as JSON. It also includes defensive handling for invalid input, duplicate identifiers, missing records, malformed JSON and file-permission problems. The application generates both individual student reports and a consolidated performance view.

The project is well aligned with a first-semester programming course because it turns basic language constructs into a coherent application. Variables, collections, functions, conditionals, loops, file handling, JSON, exception handling and modular concepts all appear in a single practical context.

PROJECT SNAPSHOT	IMPLEMENTATION
Interface	Menu-driven console application
Data persistence	JSON file stored in a writable user-data directory
Subjects	Python, Mathematics, Computer Fundamentals, Communication Skills
CRUD	Create, Read, Update, Delete student records
Performance	Total, percentage, grade, result and performance message
Validation	ID, name, course and marks validation
Technology constraint	No external Python packages required

What makes the implementation more than a basic CRUD exercise is the attention to data safety: the save routine writes to a temporary file, flushes and synchronizes it, then replaces the original JSON file. This is a small but meaningful reliability decision for a beginner project.

02 • Problem Statement, Scope & Objectives

Problem Statement

Manual student record management can be time-consuming and may result in duplicate records, data-entry errors and difficulty retrieving information. The project addresses this by providing a simple computerized system for storing and managing student information.

Project Scope

- Student record management and retrieval.
- Searching by student ID or name.
- Updating and deleting student records.
- Marks management across four configured subjects.
- Total and percentage calculation.
- Grade calculation and pass/fail determination.
- Individual and all-student performance reporting.
- Local JSON-based persistent storage.

- Input validation and common-error handling.

Objectives

- Apply Python programming concepts in a practical application.
- Digitally manage student records using structured data.
- Automate academic performance calculations and reporting.
- Practice validation, exception handling and persistent file storage.
- Build a foundation for later GUI, database and cloud-based extensions.

The project statement identifies teachers, faculty members, academic administrators, students and beginners learning Python as intended users. For the current console implementation, the strongest fit is educational demonstration and basic academic record management.

03 • Requirements & Feature Set

Functional Requirements

ID	Requirement	Implemented behavior
FR-01	Add Student	Creates a record after validating ID, name and course.
FR-02	View Students	Displays all stored students in a tabular console layout.
FR-03	Search Student	Searches ID or name using case-insensitive partial matching.
FR-04	Update Student	Updates name, course and year/semester while preserving unchanged fields.
FR-05	Delete Student	Shows details and requires explicit y/n confirmation.
FR-06	Enter Marks	Captures 0–100 marks for all configured subjects.
FR-07	Calculate Performance	Calculates total, percentage, grade, result and performance message.
FR-08	Generate Reports	Produces individual reports and an all-student performance summary.

Non-Functional Requirements

Quality Attribute	Design Response
Usability	Clear numbered menu and readable console sections.
Reliability	Validation, controlled exceptions and safer JSON replacement.
Maintainability	Functions are separated by responsibility even in the current single-file version.
Portability	Uses Python standard library and platform-aware writable data paths.
Resource efficiency	Lightweight local JSON storage with no external services.

04 • System Architecture

The application can be understood as five logical layers, even though the supplied final implementation is delivered as a single Python file. This distinction matters: the *responsibilities* are separated conceptually, while the current code has not yet been physically split into modules.

Layer	Responsibility	Representative functions
Presentation	User interaction and menu	display_menu(), input()


```
{
  "student_id": "STU001",
  "name": "Example Student",
  "course": "B.Tech CSE (AIML)",
  "year": "1st Semester",
  "marks": {
    "Python": 86,
    "Mathematics": 78,
    "Computer Fundamentals": 91,
    "Communication Skills": 84
  }
}
```

The example above illustrates the structure used by the application and is not presented as an actual stored student record.

06 • Academic Performance Engine

The performance subsystem is one of the clearest examples of reusable Python functions in the project. Marks are stored in a dictionary, and the application derives all performance metrics from that dictionary rather than duplicating calculations in multiple places.

Configured Subjects

No.	Subject	Maximum
01	Python	100
02	Mathematics	100
03	Computer Fundamentals	100
04	Communication Skills	100

Calculation Rules

Metric	Rule
Total	Sum of all entered subject marks.
Percentage	$(\text{Total} / (\text{number of subjects} \times 100)) \times 100$
Grade A	90% and above
Grade B	75% to below 90%
Grade C	60% to below 75%
Grade D	50% to below 60%
Grade F	Below 50%
Result	Pass only when every subject mark is at least 40.

Example Calculation

```
Python = 86
Mathematics = 78
Computer Fundamentals = 91
Communication Skills = 84

Total = 339 / 400
Percentage = 84.75%
Grade = B
Result = Pass
```

The source also produces a qualitative performance message: Excellent Performance, Very Good Performance, Good Performance, Average Performance, or Needs Improvement, based on the same percentage bands.

07 • User Workflow & Interface

The program presents a simple nine-option menu. The interface is intentionally console-based so that the project remains focused on Python programming concepts rather than graphical UI development.

```
=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
```

2. View All Students
 3. Search Student
 4. Update Student
 5. Delete Student
 6. Enter / Update Marks
 7. Generate Student Report
 8. Generate All Performance Reports
 9. Exit
- =====

Core Workflow

Stage	User action	System response
1	Launch application	Loads existing JSON data and displays welcome screen.
2	Select menu option	Routes control to the corresponding function.
3	Provide data	Validates input and rejects invalid values.
4	Modify records	Updates in-memory data and saves it to JSON when applicable.
5	Request report	Calculates and displays academic performance.
6	Exit	Terminates the main loop cleanly.

Search Design

Search accepts either a student ID or name and performs a case-insensitive partial match. This makes the feature more forgiving than exact matching and is particularly convenient in a console application.

08 • Validation, Reliability & Error Handling

Input validation is implemented before records are modified. Student IDs cannot be empty or contain spaces. Names must contain alphabetic characters and spaces only. Courses cannot be empty. Marks are accepted only when they can be converted to a number between 0 and 100.

Reliability Measures

- Duplicate student IDs are rejected during creation.
- Missing students are handled during update, delete, marks entry and report generation.
- Deletion requires explicit confirmation.
- Invalid numeric input for marks is caught with `ValueError` handling.
- Malformed JSON does not crash the application; the loader falls back to an empty database.
- Permission problems during read/write are caught and explained.
- The safe temporary-file save strategy reduces the risk of corrupting the main JSON file.

A particularly good design detail for a beginner project is that error handling is not limited to `input()` calls. The file layer also anticipates real-world failures such as permissions and malformed persistent data.

09 • Testing & Verification

The README specifies a unittest-based testing approach and provides the intended command:

```
python -m unittest discover tests -v
```

The documented repository structure includes separate test files for student operations, marks and validation. However, the supplied final source file is the primary implementation artifact available here, and the report does not claim a

particular number of passing tests unless supported by an actual test run.

Recommended Verification Matrix

Test Area	Expected behavior
Valid student creation	Record is stored and persisted.
Duplicate ID	Creation is rejected.
Invalid name	Input is rejected.
Invalid marks	Values outside 0–100 or non-numeric input are rejected.
Search	Matching ID/name records are displayed.
Update	Selected fields change while blank fields remain unchanged.
Delete confirmation	Only 'y' performs deletion.
Grade boundaries	90/75/60/50 thresholds map correctly.
Pass/fail	Any subject below 40 produces Fail.
Empty marks	Report indicates marks are not available.
Malformed JSON	Application warns and starts with empty data.

For the final GitHub submission, include a terminal screenshot showing the actual test command and output. Evidence is more persuasive than adjectives, a lesson humanity keeps rediscovering with impressive reluctance.

10 • Design Decisions & Learning Outcomes

Design Decisions

Decision	Reason
Console interface	Keeps the first version focused on Python fundamentals and program logic.
JSON persistence	Simple local persistence without installing or configuring a database server.
Platform-aware data path	Avoids writing into protected project directories on Windows.
Safe file replacement	Reduces the risk of corrupting the primary JSON file during writes.
Function-based design	Makes validation, calculations and CRUD logic reusable and understandable.
Standard library only	Improves portability and keeps setup simple.

Learning Outcomes

- Variables, data types, operators, conditionals and loops.
- Functions and reusable program logic.
- Lists, dictionaries and structured records.
- File handling and JSON serialization.
- Exception handling and defensive programming.
- CRUD thinking and basic data management.
- Input validation and business rules.
- Git and GitHub as part of project development and presentation.

The most important conceptual progression is from *writing code that runs* to *designing code that behaves predictably*. Validation, persistence, error handling and reporting are all steps in that direction.

11 • Future Development Roadmap

The project statement identifies several natural extensions. These can be viewed as a progression from a local educational tool toward a fuller academic management system.

Phase	Enhancement	Value
01	Modularize into separate .py files	Improves maintainability and aligns implementation with the repository structure.
02	Tkinter GUI	Improves accessibility and visual usability.
03	SQLite database	Supports structured relational persistence and larger datasets.
04	Login & roles	Separates administrative and student access.
05	Attendance management	Adds another major academic record dimension.
06	PDF report export	Produces shareable academic reports.
07	Visualization & analytics	Makes performance trends easier to interpret.
08	Cloud storage	Enables centralized access across devices.

Recommended engineering direction

The most useful next structural improvement is to split the current single-file implementation into the modules already described by the README: student management, marks, reports, validation, file handling and menu control. The behavior can remain the same while the code becomes easier to test, explain and extend.

12 • Conclusion

The Student Management System is a focused Python project that converts fundamental programming concepts into a practical academic application. It manages student records through CRUD operations, captures marks across four subjects, calculates performance, generates reports and persists information in JSON.

Its strongest technical qualities are not complexity but clarity and defensive design. The source validates inputs, prevents duplicate IDs, handles missing and malformed data, uses a platform-aware storage location and applies a safer temporary-file replacement strategy for persistence.

The project also has a clear path for growth. A modularized version can become a Tkinter application, migrate to SQLite, introduce authentication, add attendance and export professional reports. That makes the current system a sensible first version rather than a dead-end exercise.

Project: Student Management System using Python

Author: Harsh Vardhan

Course: B.Tech CSE (AIML)

Purpose: Educational and academic project

Appendix • Repository & Demonstration Checklist

Recommended repository presentation

Item	Recommended content
README.md	Project overview, features, setup, menu, screenshots and future enhancements.
Source code	Final Python implementation with clear comments and consistent naming.
statement.md	Problem statement, scope, users and feature definition.
tests/	Unit tests for student operations, marks and validation.
screenshots/	Main menu, student creation, search, marks, report and test execution.
LICENSE	Project licensing information.

Suggested demonstration sequence

- Start the application and show the welcome screen.
- Add one student with valid details.
- Attempt a duplicate ID to demonstrate validation.
- View and search the student.
- Update the course or semester.
- Enter marks for all four subjects.
- Generate the individual report.
- Generate all performance reports.
- Run the test suite and capture the output.

This sequence demonstrates the majority of the project's important behaviors in a compact live presentation.